

SDS 软件设计说明书

项目名称：基于大模型的建筑能源智能管理与运维优化系统

文档版本：V1.0

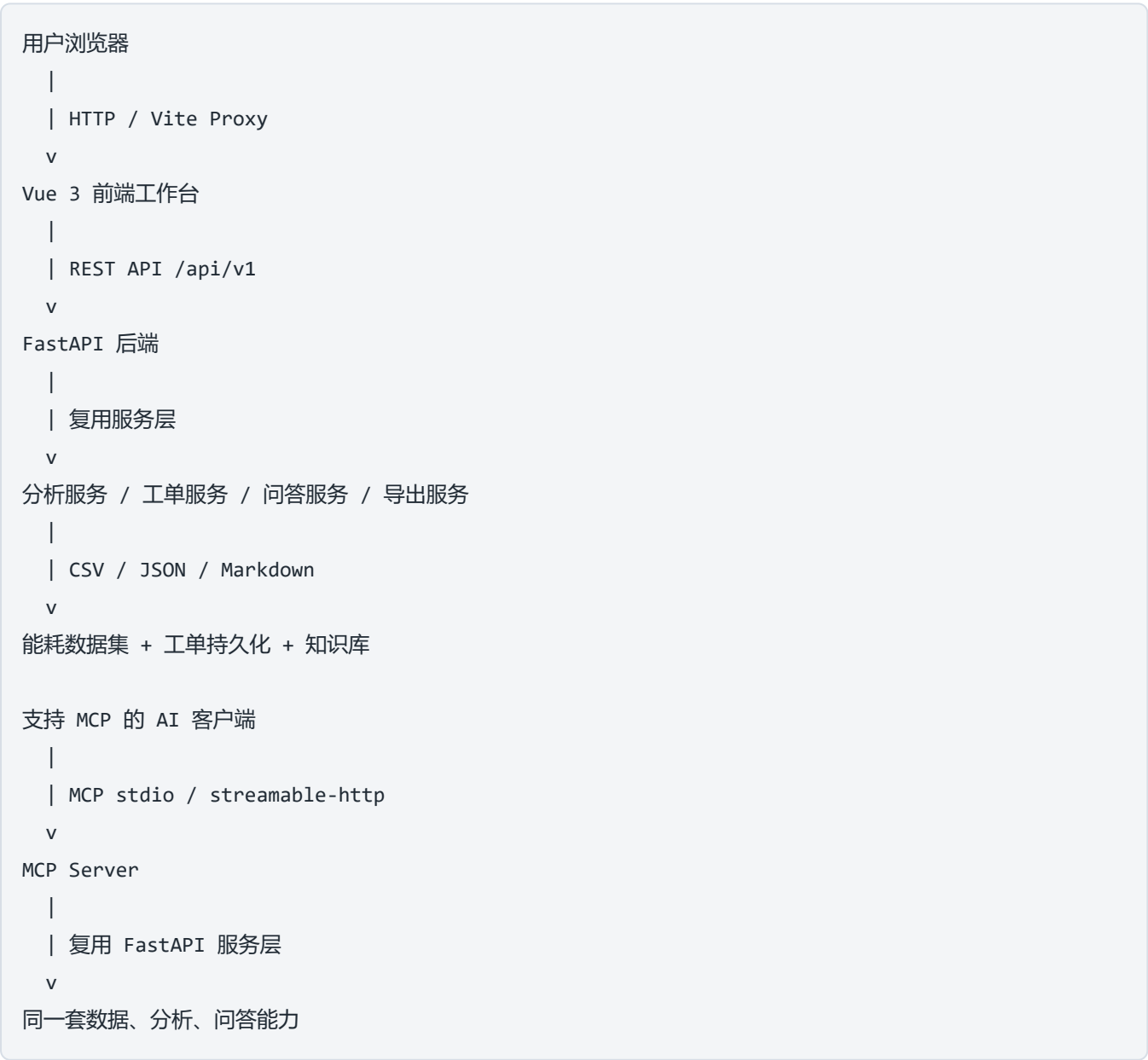
编写日期：2026-05-31

1. 文档目的

本文档说明系统总体架构、模块划分、数据设计、接口设计、MCP 设计、前端交互设计、部署方案和关键技术决策，用于支撑课程项目最终提交中的 SDS 要求。

2. 总体架构

系统采用前后端分离架构，并额外提供 MCP Server 作为面向 AI 客户端的数据接入层。



3. 技术选型

层次	技术	选择理由
后端框架	FastAPI	轻量、接口文档自动生成、测试方便
数据处理	Pandas	适合课程规模 CSV 数据分析
前端框架	Vue 3 + Vite	开发效率高，适合快速构建演示工作台
图表	ECharts + CSS 3D	兼顾统计图表和三维风险态势展示
协议扩展	MCP Python SDK	满足基于 MCP 的数据接入与查询要求
问答接口	OpenAI-compatible API	可适配多家外部大模型供应商
测试	pytest + Vite build	覆盖后端逻辑、MCP 集成和前端构建

4. 后端设计

4.1 目录结构

```
backend/  
|-- app/  
|   |-- api/  
|   |   |-- router.py  
|   |   |-- routes/  
|   |-- core/  
|   |-- schemas/  
|   |-- services/  
|   |-- main.py  
|   |-- mcp_server.py  
|-- tests/
```

4.2 API 路由层

API 路由层负责参数接收、HTTP 响应和错误处理，不直接写复杂业务逻辑。

文件	职责
routes/data.py	总览、建筑清单、数据集元信息、记录查询
routes/analytics.py	时间汇总、异常、楼层、设备、报告等分析接口
routes/work_orders.py	持久化工单增删改查

文件	职责
routes/export.py	CSV 导出
routes/assistant.py	智能问答和模型供应商信息

4.3 服务层

服务层复用给 REST API 和 MCP Server，保证 Web 页面和 AI 客户端看到的数据口径一致。

文件	职责
analysis_service.py	数据加载、异常识别、汇总分析、解释、报告
work_order_store.py	JSON 工单持久化
assistant_service.py	本地知识库问答和外部 LLM 调用编排
llm_client.py	OpenAI-compatible 模型调用封装
export_service.py	CSV 导出

4.4 配置设计

配置从仓库根目录 `.env` 读取，关键字段包括：

- `DATA_FILE`：能耗数据文件路径。
- `KNOWLEDGE_BASE_DIR`：知识库目录。
- `WORK_ORDER_FILE`：工单持久化 JSON 文件。
- `LLM_ENABLED`：是否启用外部大模型。
- `LLM_PROVIDER`、`LLM_BASE_URL`、`LLM_MODEL`、`LLM_API_KEY`：外部模型配置。

真实 `.env` 不提交，`.env.example` 只保留占位。

5. 数据设计

5.1 能耗记录

核心数据表为 `data/samples/energy_records.csv`。系统基于字段推导建筑、楼层、区域、设备状态、COP、异常规则等分析结果。

主要字段类别：

- 标识字段：记录 ID、建筑 ID、建筑名称。
- 时间字段：采集时间、日期、时段。
- 能耗字段：总电耗、空调电耗、水耗、冷量等。
- 环境字段：室外温度、湿度、人员密度。

- 设备字段：设备 ID、设备状态、设备类型。
- 派生字段：楼层、区域、COP、异常标识、异常原因。

5.2 工单数据

工单存储在 `data/runtime/work_orders.json`，运行时自动创建。该路径已被 `.gitignore` 忽略，避免把个人演示过程中的工单状态提交到仓库。

工单字段：

- `id`：工单编号。
- `record_id`：关联异常记录。
- `building_id`、`building_name`、`floor_label`：定位信息。
- `assignee`：负责人。
- `status`：处理中、已完成、已忽略。
- `note`：处理备注。
- `created_at`、`updated_at`：时间戳。

5.3 知识库数据

知识库以 Markdown 文件组织，分为：

- `manuals/`：运维手册和异常诊断指南。
- `glossary/`：指标解释和规则说明。
- `faq/`：演示问答、评估问题和检索卡片。

6. 异常分析设计

异常识别以可解释规则为主，适合课程演示和答辩说明。核心思路：

- 建筑基线：按建筑统计电耗均值和标准差，识别明显偏高记录。
- COP 规则：COP 过低时提示制冷效率异常。
- 设备状态规则：设备故障、离线、维护等状态触发异常。
- 楼层聚合：将记录异常聚合到楼层风险。
- 解释生成：对每条异常输出触发规则、指标证据、可能原因和处理建议。

这种设计优点是结果可解释、无需训练模型、可稳定复现；缺点是规则依赖人工设定，后续可升级为机器学习或时间序列预测模型。

7. 前端设计

7.1 页面结构

前端主页面为 `DashboardView.vue`，采用导航切换式工作台。

导航	主要内容
总览	KPI、三维楼宇风险态势、总体趋势
数据浏览	多条件查询、记录表格、导出
统计分析	建筑/楼层筛选、异常明细、异常解释、楼层设备分析
工单中心	创建工单、状态更新、处理闭环
决策报告	运营日报、预测、建议动作
AI 助手	知识库问答、外部模型问答

7.2 交互设计

- 三维楼层点击后联动统计分析筛选。
- 数据浏览保持独立，不受三维视图联动影响，便于全量查询。
- 健康楼层只展示健康提示，避免无异常时出现空白模块。
- 工单状态使用颜色和排序体现优先级：处理中置顶，已完成靠后。
- 外部模型不可用时回退到本地知识库回答，保证演示不中断。

8. MCP 设计

MCP Server 位于 `backend/app/mcp_server.py`，通过工具和资源暴露项目能力。

8.1 MCP Tools

主要工具包括：

- 数据集元信息查询。
- 建筑清单查询。
- 能耗记录查询。
- 概览指标查询。
- 时间汇总。
- 建筑对比。
- COP 排名。
- 异常列表。
- 异常解释。
- 楼层汇总。
- 设备汇总。
- 运营报告。
- 知识库检索。
- 智能问答。

8.2 MCP Resources

资源包括数据集元信息、建筑清单、运营报告和知识库入口。MCP 详细说明见 `docs/16-mcp-integration.md`。

9. 部署与运行设计

9.1 本地开发部署

后端：

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r backend\requirements.txt
.\scripts\start-backend.ps1
```

前端：

```
cd frontend
npm install
npm run dev
```

或使用：

```
.\scripts\start-frontend.ps1
```

MCP：

```
.\scripts\start-mcp.ps1
```

9.2 演示访问地址

- 前端： `http://127.0.0.1:5173`
- 后端文档： `http://127.0.0.1:8000/docs`
- 健康检查： `http://127.0.0.1:8000/api/v1/health`
- MCP HTTP 模式： `http://127.0.0.1:8765/mcp`

10. 安全设计

- `.env` 被 `.gitignore` 忽略。
- `.env.example` 不包含真实密钥。
- 外部模型测试脚本只输出 `OK / FAIL / SKIP`，不打印 Key。
- 工单运行期数据不提交，避免演示状态污染仓库。

- REST 与 MCP 共用服务层，减少重复实现带来的安全和一致性问题。

11. 测试设计

测试分为三层：

- 后端自动化测试：接口、服务、外部模型封装、工单、MCP。
- 前端构建测试：`npm run build`。
- 人工演示测试：按用户手册走完整路径。

一键检查脚本：

```
.\scripts\check-project.ps1
```

当前基线：后端 `75 passed`，前端构建通过。

12. 设计取舍

方案	当前选择	理由
数据库 vs CSV	CSV + JSON	课程演示更轻量，部署门槛低
机器学习异常检测 vs 规则检测	可解释规则	稳定、透明、便于答辩说明
真实传感器接入 vs 样例数据	样例数据	满足课程项目范围，避免外部硬件依赖
单 REST API vs REST + MCP	REST + MCP	满足 Web 演示和 AI 客户端接入双场景
全量外部 LLM 依赖 vs 本地回退	本地知识库优先	防止 API 不可用影响演示

13. 后续扩展

- 接入真实数据库和传感器数据。
- 增加用户登录、权限和审计日志。
- 引入时间序列预测模型和异常检测模型。
- 将工单系统扩展为多用户协同。
- 为 MCP HTTP 模式补充可视化客户端配置示例。

? PDF ?????????? Markdown ??????